

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Deep Learning

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BL3)

QUESTIONS: 1

Duration: 1 Hour
Total Marks:50

Annexure A

Error Analysis Task

- A neural network predicts student scores with an MSE of 0.85 during training and 5.20 during testing. Analyse the possible reasons for this difference and suggest suitable corrective measures.

Code of Conduct:

I Kumaran.M(192525022) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:kumaran

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5)	Level 3 – Proficient (4)	Level 2 – Developing (2-3)	Level 1 – Beginning (0-1)
Error Identification	30%	Correctly identifies all errors and their causes	Identifies most errors with minor omissions	Identifies some errors but misses key issues	Unable to identify errors

Error Analysis & Reasoning	30%	Provides clear and logical explanation of why errors occurred	Reasoning mostly corrects with minor gaps	Limited reasoning and partial explanation	No meaningful analysis
Correction Strategy	25%	Suggests appropriate corrections with justification	Suggests correct corrections with minor gaps	Partially correct corrections	Incorrect or no corrections
Interpretation of Results	15%	Clearly explains impact of errors on model performance/results	Basic interpretation with minor omissions	Limited interpretation	No interpretation provided

Answers

Analysis of the Difference Between Training and Testing MSE

- A neural network is used to predict student scores. During training, the model achieves a Mean Squared Error (MSE) of 0.85, whereas during testing the MSE increases significantly to 5.20. This large difference indicates that the model performs very well on the training data but fails to maintain the same level of accuracy on unseen data. Such a situation generally indicates poor generalization, with overfitting being the most likely cause.

1. Overfitting

- Overfitting occurs when the neural network learns not only the actual relationships between input features and student scores but also the random noise and irrelevant patterns present in the training data. As a result, the model memorizes the training examples instead of learning general rules. Consequently, it achieves a very low training error (0.85) but produces a much higher testing error (5.20). This is the most common reason for a large gap between training and testing performance.

2. Excessive Model Complexity

- A neural network with many hidden layers, a large number of neurons, or too many trainable parameters can model highly complex relationships. While this may reduce the training error, it also increases the risk of fitting unnecessary details in the training data. If the complexity of the network is not appropriate for the dataset size, the model loses its ability to generalize well.

3. Insufficient Training Data

- The quality and quantity of training data play a crucial role in the performance of a neural network. If the dataset contains only a limited number of student records, the model may not be exposed to enough variations in student performance. As a result, it cannot accurately predict scores for students whose characteristics differ from those in the training data.

4. Poor Data Quality

- Training data may contain missing values, incorrect records, inconsistent grading patterns, or extreme outliers. These issues introduce noise into the learning process. The neural network may mistakenly treat these irregularities as meaningful patterns, leading to poor prediction accuracy when evaluated on clean test data.

5. Improper Data Splitting

- The training and testing datasets should represent the same population. If the training set mainly contains high-performing students while the testing set contains a broader range of students, the model will struggle to make accurate predictions. A non-random or biased train-test split often results in higher testing error.

6. Lack of Regularization

- Regularization techniques help prevent the neural network from becoming too dependent on the training data. If techniques such as L1 regularization, L2 regularization, or dropout are not used, the network can easily memorize the training samples, resulting in overfitting and poor testing performance.

7. Excessive Training (Too Many Epochs)

- Training the neural network for too many epochs allows it to continue reducing the training error even after it has already learned the important patterns. Beyond a certain point, the model starts memorizing the training data rather than improving its understanding of the problem, causing the testing error to increase.

8. Poor Feature Selection

- The input features used for prediction may not contain sufficient information about student performance. At the same time, irrelevant or redundant features may confuse the model and reduce its ability to generalize. Proper feature engineering is therefore essential.

9. Inadequate Hyperparameter Selection

- Hyperparameters such as learning rate, batch size, number of hidden neurons, activation functions, optimizer, and number of epochs greatly influence model performance. Poorly chosen hyperparameters can cause unstable learning, slow convergence, or excessive overfitting.

Suitable Corrective Measures

1. Apply Regularization

- Use L1 and L2 regularization to penalize excessively large weights. This encourages the model to learn simpler and more general patterns. Adding dropout layers, which randomly deactivate neurons during training, also prevents the network from relying too heavily on specific neurons.

2. Reduce Model Complexity

- If the neural network contains many hidden layers or neurons, simplify the architecture by reducing their number. A smaller model is often sufficient for predicting student scores and is less likely to overfit.

3. Use Early Stopping

- Monitor the validation error during training. When the validation error starts increasing while the training error continues decreasing, stop the training process. This prevents the network from memorizing the training data.

4. Increase the Training Dataset

- Collect additional student records from different classes, schools, or academic years. A larger and more diverse dataset enables the model to learn generalized relationships rather than memorizing individual examples.

5. Improve Data Quality

Perform data cleaning by:

- Removing duplicate records.
- Handling missing values appropriately.

- Detecting and treating outliers.
- Correcting inconsistent data entries.
- Ensuring accurate labels for student scores.

Cleaner data improves both training and testing performance.

6. Normalize and Preprocess the Data

- Scale input features using normalization or standardization so that all features contribute equally during learning. Proper preprocessing helps the optimizer converge faster and improves prediction accuracy.

7. Perform Feature Engineering

- Select only the most relevant features affecting student performance, such as attendance, previous academic results, study hours, assignments, and participation. Remove irrelevant features that add noise to the learning process.

8. Use Cross-Validation

- Instead of evaluating the model using a single train-test split, use k-fold cross-validation. This provides a more reliable estimate of the model's generalization ability and reduces evaluation bias.

9. Tune Hyperparameters

Experiment with different:

- Learning rates
- Batch sizes
- Number of hidden layers
- Number of neurons

- Activation functions (ReLU, Sigmoid, Tanh)
- Optimizers (Adam, SGD, RMSProp)
- Number of training epochs

Hyperparameter tuning helps identify the best-performing model configuration.

10. Monitor Validation Performance

- Keep a separate validation dataset during training. Comparing training, validation, and testing errors helps detect overfitting early and ensures the model generalizes well.

